

Как разогнать PHP-движок слотов с 5 000 до 300 000+ спинов в секунду и сэкономить миллионы на инфраструктуре

Содержание

1. **Введение** (Окружение, базовая игра, ссылки на GitHub/LinkedIn)
2. **Описание проблем** (Специфика расчетов, легаси, Монте-Карло как Bottleneck)
3. **Архитектурные решения для Ultra Performance на PHP** (Убийство ООП, плоские массивы, Short-Circuit, микро-оптимизации)
4. **Граница разумного: Темная магия оптимизации** (Битмаски, процедурный хаос, предвычисленные карты комбинаций)
5. **Миграция на Go (Golang) за 3 недели** (Строгая типизация, оптимизация слайсов, миллионы RPS)
6. **Заключение** (Мифы о PHP, «механическое сочувствие» к железу, скрины бенчмарков)

Привет!

Я Senior/Staff Software Engineer с более чем 11-летним опытом в backend/frontend-разработке, проектировании систем и enterprise-архитектуре. Мой профиль — высоконагруженные системы с тяжелым legacy, сложной бизнес-логикой и рефакторинг монолитов с десятилетней историей «на лету», без остановки production.

Эта статья — результат глубокого анализа архитектурных проблем на сотнях реальных игр и кодовой базе, которая копилась более десяти лет. Я полностью переписал игровое ядро с нуля, вынеся всю логику взаимодействия с платформой за его пределы. Движок стал чистой математикой.

Ниже — подробный технический гайд о том, как заставить PHP выдавать производительность коммерческих вычислительных ядер, почему «красивый» код убивает процессор, и как инженерные решения могут сэкономить бизнесу миллионы долларов.

Исходный код и материал: [GitHub Репозиторий](#)

Связь со мной: [LinkedIn Профиль](#) — открыт для предложений и идей.

Окружение и характеристики испытательного стенда

Все бенчмарки и показатели производительности, о которых пойдет речь, зафиксированы на одном и том же ноутбуке:

- **ОС:** Debian GNU/Linux 12 (bookworm) — Ядро Linux 6.1.0-44-amd64 (x86_64)
- **CPU:** 12th Gen Intel(R) Core(TM) i7-1260P (12 ядер / 16 потоков, до 4.70 GHz)
- **RAM:** 32 GB
- **NVME:** Samsung SSD 990 PRO 1TB
- **Базовая игра:** Для чистоты эксперимента взята простая модель — игровое поле 3x3, 7 символов, 3 выигрышные линии (горизонтали слева направо). Никаких фриспинов, респинов, бонусных раундов и дополнительных механик.

Давайте начистоту: конечному клиенту абсолютно плевать, насколько красивый код написан у вас в репозитории. Игра крутится, слоты мигают, баланс сходится — для него система работает. На коленке можно писать и 20 лет, но для бизнеса внутренняя архитектура — это либо ракетное топливо, либо бетонная плита на ногах.

Мне пришлось столкнуться с кодовой базой крупного iGaming-проекта со шлейфом из сотен игр, которая представляла собой хрестоматийный музей технического долга, вот малая часть:

- **Математика во float:** Все расчеты велись в плавающей точке, генерируя классические проблемы округлений и расчетов.
- **Версионирование через «простыни» switch-case:** для поддержки старых игр.
- **GodRepository:** все игры и инструменты в едином месте
- **Файлы-монстры:** Один файл мог раздуваться до десятков тысяч строк кода, где сосуществовали по 10 функций с диким неймингом, которые делали абсолютно одно и то же, но «каждая по-своему».
- **Вакуум вместо тестов и документации:** Кодовая база передавалась как легенды и мифы древней Греции — из уст в уста.

Когда у компании в портфеле сотни игр, а на платформе одновременно крутят барабаны десятки тысяч игроков, нагрузка на сервера становится колоссальной. Это не классический веб-CRUD. В стандартном вебе процессор в основном простаивает, ожидая ответа от базы данных или сети. Если там нужно убрать оверхед на инициализацию PHP-приложения на каждый запрос, мы просто берем что-то вроде RoadRunner или Swoole, которые держат воркеры в памяти.

Но в iGaming сервер непрерывно занят чистыми, тяжелыми вычислениями игровой логики на каждый чих игрока.

И вся эта история сильно греет CPU, особенно когда нужно провести **Монте-Карло эмуляции** — обчислать 10 000 000 000 (10 миллиардов) спинов или других взаимодействий игрока с игрой для каждого RTP профиля. Это обязательный этап для валидации математики, расчета волатильности (Coefficient of Variation) и верификации RTP перед отправкой софта в жесткие сертификационные лаборатории (iTechLabs, eCOGRA).

Если писать игровой РНР движок по академическим канонам «чистого ООП» (с объектами под каждый символ, линию и результат спина), РНР захлебнулся бы на аллокациях памяти и выдал жалкие **100–200 спинов в секунду**. Проверено на опыте.

Старый легаси-код писался во времена динозавров. Он забит тяжелыми системными функциями к примеру (`array_merge`, `array_flip`) и другими, избыточными проверками, вложенными глубокими структурами (вызывающими Cache Miss) и постоянным взаимодействием с файловой системой для сессий и логирования, его средняя производительность на эмуляции примерно **~5 000 итераций(спинов, покупки и т.д.) в секунду**.

Сухая математика бизнес-потерь

Если вам нужно прогнать базовые **10 миллиардов** итераций при скорости старого легаси-движка в 5 000 спинов/сек, мы получаем **катастрофический Bottleneck**:

$10\,000\,000\,000 / (5\,000 \times 3\,600) \approx 555,6$ **часов на одном ядре**

- На **10 ядрах** параллельно — это **55.5 часов** непрерывной работы.
- На **20 ядрах** — **27.7 часов**.

И это только один прогон. А таких циклов требуются десятки или сотни на одну игру. Ведь малейший сдвиг вероятности выпадения символа или кастомной фичи влияют не только на отдачу но и на retention, делая игру либо разорительной для оператора, либо скучной для игрока.

В итоге показатель **Time-to-Market (TTM)** растягивается на недели, разработчики и математики днями ждут результатов симуляций, а компания жжет колоссальные бюджеты на аренду многоядерных серверов в облаках.

Новое ядро изначально проектировалось так, чтобы исправить обе проблемы: сделать код читаемым, масштабируемым (где сам код и есть документация) и при этом выжать максимум производительности из железа.

Ниже представлен детальный разбор того, как снять оковы интерпретатора, избавиться от мусорных операций с памятью и заставить РНР 8.5 JIT работать на пределе физических возможностей процессора, выдавая **300 000+ итераций взаимодействия игрока с игрой (e.g. spin/buy) в секунду**.

Архитектурные решения для Ultra Performance

1. Уничтожение памяти: Убийство ООП в Hot Path

Главный враг экстремальной производительности в РНР — это выделение памяти (Memory Allocation) и сборщик мусора (Garbage Collection). Циклы генерации игровых полей, поиска линий и расчета выплат — это наш **Hot Path** (горячий участок кода), который во время симуляций выполняется миллиарды раз.

В архитектуре есть код, который делегирует, направляет и структурирует — там паттерны программирования (порождающие, структурные, поведенческие) жизненно необходимы. Но внутри Hot Path вся эта академическая красота должна умереть. Здесь код обязан быть максимально тупым, прямолинейным и примитивным, словно написанным в каменном веке.

Объекты идут под нож

Никаких объектов внутри цикла. Создание даже самого легковесного объекта вроде `new WinLine()` или `new SpinResult()` внутри Hot Path заставляет процессор тратить сотни тактов на аллокацию структуры в памяти, а затем столько же — на её последующий обход и очистку сборщиком мусора.

В горячем цикле имеют право существовать **только примитивные типы данных и плоские массивы**.

Осознанный компромисс: Да, массивы в РНР — это тоже структуры данных, и на них тоже выделяется память. Да, отказ от объектов в пользу плоских массивов превращает отладку и поддержку кода в головную боль. Но это фундаментальное ограничение самого языка, и мы сознательно идем на эту жертву.

Это компромиссное решение: вне Hot Path мы по-прежнему можем писать красивый ООП-код, использовать объекты и абстракции, но перед самой отправкой данных в горячий цикл расчетов мы обязаны полностью «раздеть» их и конвертировать в плоские, простые примитивы.

Отказ от интерфейсов, динамики и переход на ручной SOLID

Никаких полиморфных вызовов, паттернов «Стратегия» или динамических переменных вида `$$variable`. JIT-компилятор PHP 8.5 просто не способен оптимизировать код, если типы данных и пути выполнения постоянно меняются в рантайме. И процессору, и JIT-у нужен последовательный, абсолютно предсказуемый поток выполнения (flow control).

При этом мы не выбрасываем правила SOLID на помойку, но контроль за их соблюдением полностью переносится на разработчика — мы переходим на **ручное управление архитектурой**. Наша задача как инженеров — заранее заложить такую масштабируемость, чтобы под каждую новую игру не приходилось изобретать велосипед или переписывать систему с нуля.

Чистота функций внутри Hot Path становится абсолютной:

- Если метод `calculateWin()` спроектирован для расчета выигрышей, он занимается **только** расчетом выигрышей.
- В нем не может быть никакого побочного логирования.
- Он не имеет права неявно модифицировать входные данные.
- В нем полностью исключена любая побочная логика и «дичь», способная сбить оптимизацию цикла.

Запрет на клонирование

Забудьте про ключевое слово `clone`, и уж тем более про попытки глубокого клонирования через тяжелую сериализацию `serialize / unserialize`. Внутри горячего цикла это архитектурное преступление против процессора. Вся подготовка, копирование и трансформация данных должны завершиться *до* того, как сработает первая итерация.

2. Сплющивание матриц: Война с Cache Miss

Двумерные массивы — стандарт для отображения игровой сетки `Reels[X][Y]`. Но массив в PHP — это упорядоченная хэш-таблица (`zval`). Доступ к элементу `$reels[$x][$y]` заставляет процессор бесконечно прыгать по указателям. На больших объемах это приводит к **L1/L2 Cache Miss** (промахам кэша) — ядро процессора простаивает в ожидании данных из медленной RAM.

Решение: Склеить все барабаны слева направо, сверху вниз в один плоский одномерный массив. Игровая сетка 3x3 превращается в массив с индексами от 0 до 8.

- Было (2D): `$reels[reelId][rowId]`
- Стало (1D): `$reels[position]`

Для фиксированных полей координаты точки [X, Y] переводятся в плоский индекс по формуле:

`position = rowID * gameFieldWidth + reelID`

Примечание: Эта формула не работает при динамических барабанах разной высоты (там применяется другая логика индексации), но принцип плоской структуры должен сохраняться. Для поиска X, Y из позиции нам потребуется конфигурация размеров барабанов.

Теперь выигрышная линия на самом верху сетки 3x3 — это просто проверка индексов 0, 1, 2. Данные лежат в памяти последовательно и идеально оседают в L1-кэше процессора.

3. Алгоритмическое пуленепробиваемое сокращение (Short-Circuit)

Это лишь на первый взгляд кажется, что изменения незначительны. Но когда речь заходит о миллиардах операций, даже малейшая оптимизация, экономящая всего 5% ресурсов — это потенциально сэкономленные десятки или сотни тысяч долларов в зависимости от масштаба проекта.

Оптимизация конкретных алгоритмов поиска выигрышей спокойно дает кратный xN буст скорости. Чтобы этого добиться, мы полностью отказываемся от универсальных калькуляторов. Вместо этого мы создаем отдельные изолированные методы или классы, заточенные под одну-единственную задачу: найти выигрыши одним конкретным способом.

В моем коде алгоритмы жестко разделены по классам. Они **прогреваются ровно один раз** на самом первом спине эмуляции и сохраняют в локальных параметрах объекта индексированный кэш и выделенную память, которая в дальнейшем просто обновляется. Таким образом, уже на втором спине калькулятор заранее знает высоту каждого барабана, длину поля и количество возможных выигрышей. Вся рабочая память под это дело уже выделена, закеширована и индексирована.

В качестве примера возьмем калькулятор линий. Вместо того чтобы в лоб перебирать все возможные комбинации на поле, мы применяем агрессивный ранний выход из циклов (Short-Circuit):

- **Проверка линий вместо комбинаций:** Если в игре всего 5 выигрышных линий, а потенциальных комбинаций символов — 25, мы пишем цикл строго по 5 линиям. Это экономит десятки итераций на каждом одиночном спине. На дистанции в 1 миллиард спинов (1 ккк) это чистая экономия в **20 миллиардов итераций, Карл!**
- **Вход со второго барабана:** Если по правилам игры комбинации считаются строго слева направо и должны содержать минимум 3 символа, то проверку линии можно начинать сразу со второго барабана (индекс 1). Мы берем символ на второй позиции и смотрим налево (на первый барабан, индекс 0). Если символ слева не совпадает и при этом не является Вайлдом — линия обрывается сразу же (**continue/break**). Мы выполнили на одну операцию меньше, даже не двигаясь дальше по линии. Если символ совпал — только тогда смотрим направо.

RHP

```
// Концепт ультра-быстрой проверки линии из 3 символов (позиции 0, 1, 2)
```

```
if ($flatReels[1] !== $flatReels[0]) {  
    continue; // Линия оборвана, первый и второй символы не сошлись  
}  
if ($flatReels[1] !== $flatReels[2]) {  
    continue; // Линия оборвана, второй и третий не сошлись  
}  
// Если дошли сюда — у нас зафиксирован выигрыш из 3 символов
```

Для понимания работы на плоской структуре: мы проходимся по каждой выигрышной линии. Допустим, у нас есть линия, занимающая позиции **4, 5, 6**. Мы берем её второй элемент (по факту это индекс 5 в плоском массиве барабанов) и сравниваем его с соседними элементами этой линии на позициях 4 и 6.

Конечно, это упрощенный концепт. В реальных играх линии могут быть и по 12 символов, а правила поиска — значительно сложнее. Например, когда нужно выделить строго самый высокооплачиваемый символ, найти самую длинную комбинацию, или обработать ситуацию, когда на линии одновременно выпали 3 Вайлда и 3 одинаковых символа, и платить должны оба варианта. Это усложняет итоговый код, но саму суть оптимизации и инкапсуляции алгоритма не меняет.

В примере выше мы экономим как минимум 1 итерацию на каждую линию при игровом поле 3x3. Казалось бы, всего одна итерация. Но на 1 миллиарде эмуляций для игры с 3 линиями мы экономим **3 миллиарда итераций** и абсолютно всю логику, которая должна

была выполняться внутри них. В случае старого кода это были бы миллиарды обращений к хэш-таблицам внутри двумерных массивов, которые дергали память, собирали Cache Miss и заставляли движок тупить. Наносекунды превращаются в часы, а операции взаимодействия с памятью — в реальную экономию миллионов.

Гибкая логика Вайлдов

Вместо тяжелых и развесистых проверок «в лоб» внутри циклов, логика заменяется на простую конфигурационную карту — **Lookup Table (LUT)**. В ней для каждого типа Вайлда заранее прописан жесткий массив взаимодополняемых символов:

PHP

```
$wildMap = [  
    'WILD_V1' => ['A' => true, 'B' => true, 'C' => true],  
    'WILD_V2' => ['D' => true, 'E' => true],  
];
```

Это легко расширяемая структура. Во время поиска комбинаций мы просто проверяем по этой статической карте, присутствует ли текущий и сравниваемый символ в списке разрешенных замен. Это критично, поскольку на реальных проектах один Вайлд может не заменять Вайлд второй версии, Скаттер или какой-нибудь специфический бонусный символ. Сложная логика превращается в мгновенный поиск по ключу.

4. Память: Выдели один раз и перезаписывай

Динамическое создание массивов в цикле = аллокация + уборка. К примеру, пустое игровое поле создается **один раз** при инициализации сервиса и заполняется **null** или пустыми символами. В процессе эмуляции мы не генерируем новый массив результатов, а просто **перезаписываем** значения по существующим индексам плоской структуры.

5. Микро-оптимизации на уровне ядра PHP

В Hot Path синтаксис напрямую влияет на наносекунды:

Что делает код медленным	Как нужно писать в Hot Path	Почему это работает

<code>count(\$array)</code>	<code>\count(\$array)</code>	Вызов из глобального пространства (<code>\</code>) исключает этап поиска функции в текущем namespace. Само собой, все что можно просчитать заранее, кешируем и индексируем.
<code>array_merge()</code> , <code>array_flip()</code> и другие	Ручной итерационный <code>for</code>	Встроенные функции выделяют новую память под массив. Простой <code>for</code> идеально компилируется JIT-ом.
Использование <code>float</code> (0.05)	Использование <code>int</code> (5)	Все расчеты переводятся в scaled x100 единицы (центы/копейки). Процессор считает <code>int</code> в разы быстрее и точнее.
<code>random_int()</code>	<code>\mt_rand()</code>	<code>random_int()</code> криптографически безопасен, но слишком тяжел. На дистанции в 100 млн спинов закон больших чисел сглаживает огрехи <code>mt_rand()</code> , но скорость возрастает кратно.

Критически важно: Никакого `xDebug` в среде тестирования. Он должен быть полностью отключен (`turn off`), так как его оверхед на логирование шагов убивает производительность на корню.

6. Разделение сред: Production vs Emulation

Одна из главных ошибок — попытка запускать математическую эмуляцию на том же коде, который отдает **Provably Fair** результаты реальным игрокам на продакшене.

В **Production** режиме вам необходим честный, воспроизводимый и сертифицированный ГСЧ на базе шифрования (**HMAC-SHA256**, маскирование). Но этот функционал — **убийца скорости номер один**. В режиме **Emulation** (Монте-Карло) эта логика полностью отключается и заменяется на инлайновый `mt_rand`.

Также эмуляция требует **нулевого I/O**. Любой вывод логов, запись в файловую систему или даже банальный **echo** в консоль на каждом спине опускает скорость движка до уровня плинтуса. Запись идет исключительно порциями (например, раз в 1 000 или 10 000 спинов) для обновления прогресс-бара. По завершению просто записываем результаты в файл.

Сравнительные результаты эволюции архитектуры

В старом коде (где PHP-процессы запускались через Node.js-раннер для распределения по ядрам и убиралось в I/O) чистая скорость без вывода данных составляла в прыжке около **10 000 спинов в секунду на ядро**.

После написания абсолютно нового ядра и новых инструментов, апгрейда до **PHP 8.5** и включения **JIT** для прогрева циклов, результаты для простой игры 3x3 на 7 символов с 3 выигрышными линиями зафиксировались на следующих отметках:

Metric	Value
Iterations	10000000
Iterations per Second	304853.12
Total Execution time	00:00:32.802

Итог: 10 миллионов спинов обсчитываются всего за **32 секунды** на одном ядре. Чистая скорость составила **304 000+ спинов в секунду**. Это ускорение в **60 раз (6000%)**.

Процесс верификации математических моделей и настройки игр сократился с недель до минут. Ресурсы которые сервера расходуют на каждый спин также колосально сократились в разы. За счет таких сокращений растет производительность всей команды и соответственно прибыль бизнеса.

Граница разумного: Темная магия оптимизации

Можно ли разогнать PHP еще сильнее? Да, но каждая последующая оптимизация наносит тяжелый удар по читаемости, поддержке и масштабируемости кода:

1. **Битмаски:** Перевод расчетов линий и комбинаций на битовые маски может дать еще от 25% до 50% прироста скорости.
2. **Тотальный процедурный код:** Полный отказ от классов и перевод всего движка в один монолитный файл на чистых массивах и функциях. Код превращается в нечитаемую кашу, где в один метод передаются числовые инструкции вроде `countType => 1` (где 1 — это кастомный тип расчета Ways). Способ расчета кэшируется в переменной выше метода.
3. **Глобальное хранилище** состояния (Аналог Vuex/Redux): Создание единого хранилища памяти на самом верхнем уровне приложения. Сервисы общаются только через него, полностью исключая генерацию новых структур данных. Одни части памяти жестко кэшируются, другие (барабаны) — перезаписываются.
4. **Precomputed Win Combo Maps:** Предварительный расчет всех возможных выигрышных комбинаций на этапе инициализации игры. Например, формируется массив, где ключом является строка `[A_A_A] => pay_multiplier`. На живом спине мы просто склеиваем выпавшие символы в строку и проверяем `isset($possibleCombos[$key])`. Если есть Вайлды, придется генерировать карты вида `[W_A_A]`, `[A_W_A]`. Это требует огромного количества памяти и долгой инициализации, что делает подход абсолютно неприменимым в HTTP-запросах (Production), но жизнеспособным для эмуляции.
5. **SplFixedArray:** Для ассоциативных хэш-мап со строковыми ключами это могло бы помочь. Но для наших плоских массивов, где ключами выступают простые целочисленные индексы (0, 1, 2...), `SplFixedArray` в PHP 8.5 создает лишь лишний оверхед. Стандартный упакованный массив (packed array) под капотом ядра оптимизирован лучше.

Я сознательно остановился на отметке в 304k спинов, так как дальнейший разгон превратил бы коммерческий движок в неподдерживаемый ад, остановив разработку новых игр.

Масштабирование через Node.js: Обе версии PHP-движка (и старая, и новая) для продакшен-эмуляции используют Node.js-раннер, который запускает PHP-процессы на указанное количество ядер процессора. В старой версии связка Node.js + PHP без I/O выдавала около 10 000 спинов/сек (эффект от избавления от I/O). В новой версии показатели улетели в космос, так как оверхед на уровне самого PHP стал минимальным.

Версионирование софта: Апгрейд интерпретатора с PHP 7.4 до PHP 8.5, включение JIT для компиляции циклов Hot Path. Разделение репозиторий — один для инструментов и

ядра, и по отдельному репозиторию на каждую игру чтобы можно было дорабатывать ядро, инструменты без влияния на уже сертифицированные игры.

Эволюционный скачок: Миграция на Go за 3 недели

К сожалению, в определенный момент компанию мне пришлось покинуть. Но, я решил пойти еще дальше и полностью закрыть архитектурные недостатки, заложенные в самой природе PHP. Я перенес всю математическую логику движка на Go (Golang).

Самое интересное: **я никогда в жизни до этого не писал на Go**. Мой стек ограничивался PHP/JS и немного TypeScript в рамках Vue.js 2 (опыт с Java и C++ был слишком давно, чтобы брать его в расчет). Перенос логики занял у меня порядка 2–3 недель неполного рабочего дня.

Как это возможно? У меня уже была готовая, мною спроектированная и вылизанная архитектура, очищенная от платформенного и package мусора, и ИИ в качестве персонального ментора. Я крайне скептически отношусь к современным нейросетям из-за их постоянных галлюцинаций, бреда и откровенной лжи. Поэтому я использовал ИИ как жесткий поисковик: задавал тысячи узкоспециализированных, конкретных вопросов, как когда-то давно на StackOverflow или Хабр QA(Тостер). Количество вопросов измерялось тысячами.

Результат превзошел все ожидания.

Главный вывод из сравнения PHP и Go:

В PHP самой большой болью оставалась динамическая структура плоских массивов. Стоило случайно передать не тот индекс или не ту структуру данных при изменении логики игры — всё вылезало лишь на эмуляции, если вылезало вообще. Чего только стоит `5 < []=true` при `strict_types=1`.

Go полностью решил эту проблему на уровне компилятора:

- Строгая типизация как защита от дурака: Компилятор Go бьет разработчика по рукам, если тот пытается передать `uint8` туда, где ожидается кастомный тип

`SymbolID` (даже если под капотом это тот же `uint8`). Код либо компилируется и гарантированно работает быстро, либо не компилируется вовсе.

- Результат на Go: Использование аналогичных принципов (Data Locality, Lookup Tables[256]`DataType` под байтовые ключи символов, полное предвыделение `capacity` слайсов и выполнение расчетов strictly на стеке процессора без аллокаций в куче) позволило мне выжать миллионы спинов в секунду на одном единственном ядре (Single Core, Single Thread).

При такой скорости отпадает необходимость усложнять систему и опускаться до уровня C++. Архитектура, вобравшая в себя опыт анализа инструментов который использовались для разработки более 1300+ продакшен-игр, на Go стала монолитной, безопасной и запредельно быстрой.

Заключение

Низкая скорость РНР под высокими нагрузками — это миф, созданный теми, кто не понимает РНР. Понимание того, как интерпретатор работает с памятью (`zval`), как JIT оптимизирует плоские циклы, и как процессор использует свой кэш, позволяет писать код с «механическим сочувствием» к железу.

Да, код теряет привычную для среднестатистического веб-разработчика «красоту», но он приобретает истинную инженерную элегантность и приносит бизнесу доход за счет увеличения производительности, экономии ресурсов, сокращения и облегчения цикла разработки без потери качества.

Если статья вызовет интерес, в следующей части я распишу технические нюансы миграции на Go, покажу бенчмарки на миллионы RPS и разберу, как заставить Go работать на экстремальную производительность математических ядер.

← → × Not secure https://s7ih6lg2gls34rql.localhost/emulation/?spins=10000000&nodb=true&noParams=true
Demo - Loc cc cs

Spins - 784187/10000000

Array

```
(  
  [spin_count] => 907656  
  [win_count] => 123468  
  [win_count.standart] => 123468  
  [win_count.free] => 0  
  [win_count.bonus] => 0  
  [hit] => 7.35  
  [hit.standart] => 7.35  
  [hit.free] => 0  
  [hit.bonus] => 0  
  [win] => 3537400  
  [bet] => 3920940  
  [win.standart] => 3537400  
  [bet.standart] => 3920940  
  [prize.standart] => 7074825  
  [win.free] => 0  
  [bet.free] => 0  
  [prize.free] => 0  
  [win.bonus] => 0  
  [bet.bonus] => 0  
  [prize.bonus] => 0  
  [prize] => 7074825  
  [rtp] => 0.90218161971364  
  [rtp.standart] => 0.90218161971364  
  [rtp.free] => 0  
  [rtp.bonus] => 0  
  [transitions] => transition_to.standart-1;  
  [lastWinType] => standart  
  [free.averageBetWin] => 0  
  [bonus.averageBetWin] => 0  
  [last_positive_bet] => 5  
  [spin_count.standart] => 907656  
  [spin_count.free] => 0  
  [spin_count.bonus] => 0  
  [last_win_type] => standart  
  [maxwin] => 2050  
  [maxX] => 410  
  [maxprize] => 2050  
  [freeStart] => 0  
  [maxprize.standart] => 2050  
  [feature.base.maxprize] => 2050  
  [maxwin.standart] => 2050  
  [maxX.standart] => 410  
  [transition_to.standart] => 1  
)
```

Total execution time in seconds: 168.93

Speed - 4641.97spins/second

coreAP

docker

php index.p

conf.d/

gamesC

php.ini

```
1  error_reporting = E_ALL
2  display_errors = On
3  display_startup_errors = On
4  log_errors = Off
5  log_errors_max_len = 1024
6  ignore_repeated_errors = On
7  ignore_repeated_source = On
8  report_memleaks = On
9  #track_errors = On
10 html_errors = On
11
12 memory_limit = 4096M
13
14 upload_max_filesize = 5000M
15 post_max_size = 5000M
16 max_execution_time = 86400
17 max_input_vars = 100000
18  Speransky, 5/15/23, 9:35 AM • init
19 include_path = "./:/var/www/core"
20 default_charset = "utf-8"
21
22 opcache.enable = 1
23 opcache.memory_consumption=1024
24 opcache.max_accelerated_files=100000
25 opcache.max_wasted_percentage=10
26 opcache.validate_timestamps=1
27 opcache.revalidate_freq = 1
28 #extension=tideways_xhprof.so
29
```

Terminal

Local

+

▼

```
bonbon@bonbonnetwork:/var/www/casinoSystem$ docker compose exec php php -v
WARN[0000] /var/www/casinoSystem/docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
PHP 7.4.33 (cli) (built: Nov 15 2022 06:05:55) ( NTS )
Copyright (c) The PHP Group
Zend Engine v3.4.0, Copyright (c) Zend Technologies
    with Zend OPcache v7.4.33, Copyright (c), by Zend Technologies

bonbon@bonbonnetwork:/var/www/casinoSystem$
```

42

89

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

final class EmulationConsoleCommand extends Command

public function execute(InputInterface \$input, OutputInterface \$output): int {

// MAIN LOOP

\$loopStartedAtFloat = microtime(as_float: true);

\$coinsBet = \$gameConfig->allowedCoinsBets()[0];

\$totalWin = 0.0;

\$winFrequency = 0;

\$minSpinWin = PHP_FLOAT_MAX;

\$maxSpinWin = 0.0;

bonbon, 2/27/26, 12:25 PM • calculation service and emulation optimization / refactor

\$averageWinPerSpin = 0.0;

\$volatilityModule = 0.0;

\$totalSpined = 0;

for (\$iteration = 0; \$iteration < \$iterations; \$iteration++) {

\$roundResult = \$engine->roundResult(staticConfig: \$staticConfig, coinsBet: \$coinsBet);

\$winInfo = \$roundResult['winInfo'];

\$spinWin = \$winInfo['roundScaledWin'];

// --- core stats ---

\$totalWin += \$spinWin;

if (\$spinWin > 0) {

\$winFrequency++;

}

App\Infrastructure\ConsoleCommand > EmulationConsoleCommand > execute()

Local

+

-

10000/10000 [=====] 100%

Metric	Value
Iterations	10000000
Iterations per Second	304853.38
Total Execution time	00:00:32.802

Wins Calculations

Metric	Value	Description
RTP	135.462%	Return to Player. Long-term payout percentage to see how much money user gets back.
Volatility	6.787	Standard deviation of win per spin. Measures fluctuation intensity. Higher = larger swings.
Coefficient of Variation	5.01	Risk intensity relative to average win per spin.
Total Wins	540081	How many times user won.
Hit Rate	5.401%	Percentage of spins that produced any win > 0. How often player wins. Psychological metric.
Total Win	135462.15	Sum of all wins across all simulated spins.
Total Bet	10000000	Total wagered amount.
Minimal Win	0.1	Smallest win recorded in a single spin.
Maximal Win	0.9	Largest win recorded in a single spin (max exposure).